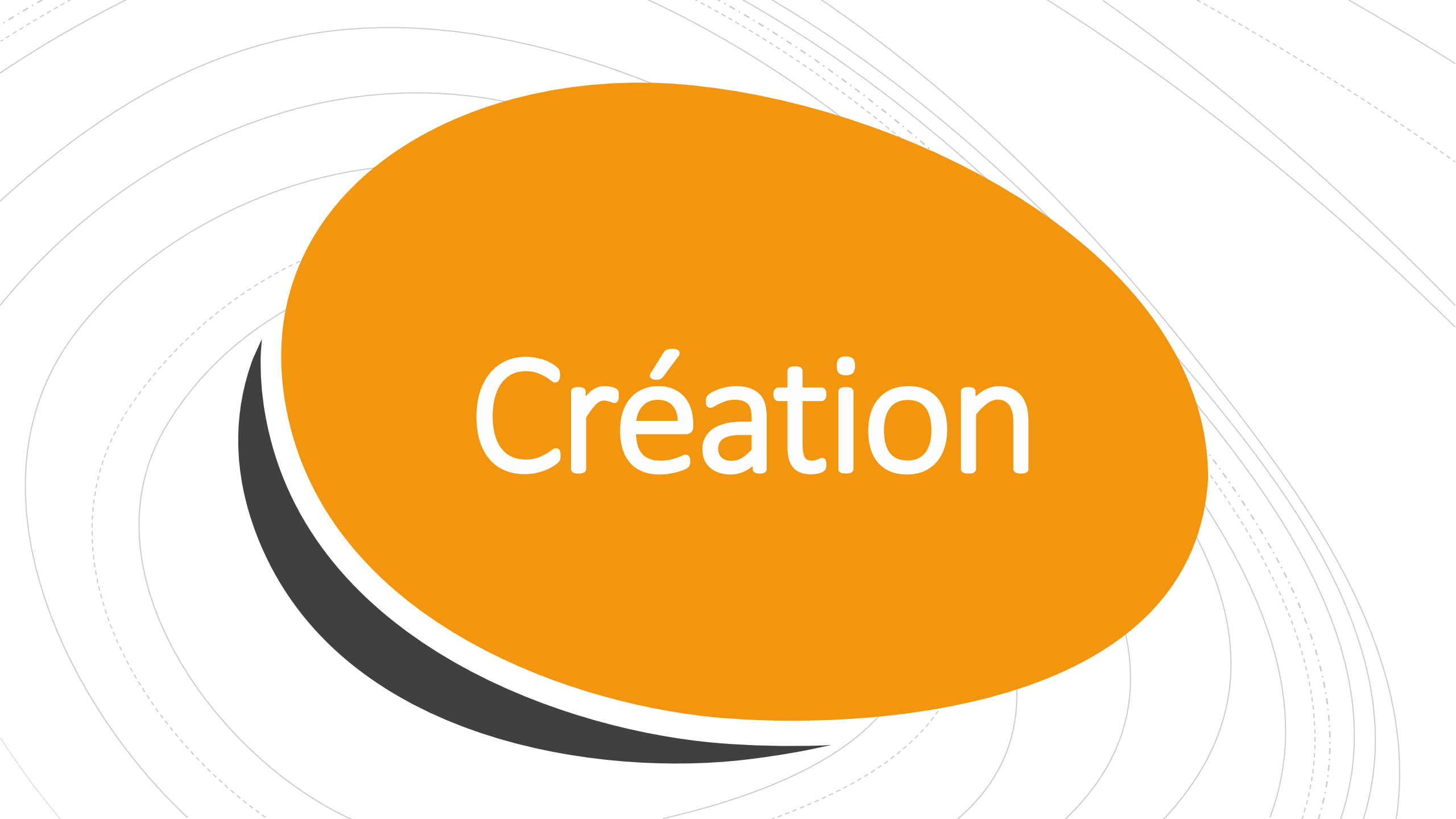


Manipulation des réponses http

The background features a series of concentric circles in light grey, some solid and some dashed, creating a sense of depth and movement. A dark grey, curved swoosh is positioned on the left side, partially overlapping the orange oval.

Création

Toutes les routes et tous les contrôleurs doivent renvoyer une réponse à renvoyer au navigateur de l'utilisateur. Laravel propose plusieurs façons différentes de renvoyer des réponses. La réponse la plus basique consiste à renvoyer une chaîne à partir d'une route ou d'un contrôleur. Le framework convertira automatiquement la chaîne en une réponse HTTP complète :

```
Route::get('/', function () {  
    return 'Hello World';  
});
```

Chaînes et tableaux

En plus de renvoyer des chaînes à partir de vos routes et de vos contrôleurs, vous pouvez également renvoyer des tableaux. Le framework convertira automatiquement le tableau en une **réponse JSON** :

```
Route::get('/', function () {  
    return [1, 2, 3];  
});
```

Chaînes et tableaux

En règle générale, vous ne renverrez pas simplement de simples chaînes ou des tableaux à partir de vos actions de routage. Au lieu de cela, vous renverrez des instances **Illuminate\Http\Response** ou des **vues complètes** .

Le renvoi d'une instance complète **Response** vous permet de personnaliser le code d'état HTTP et les en-têtes de la réponse. Une instance **Response** hérite de la classe **Symfony\Component\HttpFoundation\Response**, qui fournit diverses méthodes pour créer des réponses HTTP :

```
Route::get('/home', function () {  
    return response('Hello World', 200)->header('Content-Type', 'text/plain');  
});
```

Objets de réponse

Vous pouvez également renvoyer des modèles et des collections **ORM Eloquent** directement à partir de vos routes et de vos contrôleurs.

Lorsque vous le faites, Laravel convertira automatiquement les modèles et collections en **réponses JSON** tout en respectant les attributs cachés du modèle :

```
use App\Models\User;

Route::get('/user/{user}', function (User $user) {
    return $user;
});
```

Modèles et collections éloquent

Gardez à l'esprit que la plupart des méthodes de réponse peuvent être chaînées, ce qui permet la construction fluide d'instances de réponse.

Par exemple, vous pouvez utiliser la méthode **header** pour ajouter une série d'en-têtes à la réponse avant de la renvoyer à l'utilisateur :

```
return response($content)
    ->header('Content-Type', $type)
    ->header('X-Header-One', 'Header Value')
    ->header('X-Header-Two', 'Header Value');
```

Attacher des en-têtes aux réponses

Ou, vous pouvez utiliser la méthode **withHeaders** pour spécifier un tableau d'en-têtes à ajouter à la réponse :

```
return response($content)
    ->withHeaders([
        'Content-Type' => $type,
        'X-Header-One' => 'Header Value',
        'X-Header-Two' => 'Header Value',
    ]);
```

Attacher des en-têtes aux réponses

Vous pouvez attacher un cookie à une instance **Illuminate\Http\Response** sortante à l'aide de la méthode **cookie**. Vous devez transmettre le nom, la valeur et le nombre de minutes pendant lesquelles le cookie doit être considéré comme valide à cette méthode:

```
return response('Hello World')->cookie(  
    'name', 'value', $minutes  
);
```

Attacher des cookies aux réponses

Si vous souhaitez générer une instance

Symfony\Component\HttpFoundation\Cookie pouvant être attachée à une instance de réponse ultérieurement, vous pouvez utiliser le Helper global **cookie**. Ce cookie ne sera pas renvoyé au client sauf s'il est attaché à une instance de réponse :

```
$cookie = cookie('name', 'value', $minutes);  
  
return response('Hello World')->cookie($cookie);
```

Génération d'instances de cookies

Vous pouvez supprimer un cookie en le faisant expirer via la méthode **withoutCookie** d'une réponse sortante :

```
return response('Hello World')->withoutCookie('name');
```

Expiration anticipée des cookies

The background features a series of concentric circles in light grey, some solid and some dashed, creating a sense of depth and movement. A dark grey, curved swoosh or arc is positioned on the left side, partially overlapping the orange oval.

Redirection

Les réponses de redirection sont des instances de la classe **Illuminate\Http\RedirectResponse** et contiennent les en-têtes appropriés nécessaires pour rediriger l'utilisateur vers une autre URL. Il existe plusieurs façons de générer une instance **RedirectResponse**. La méthode la plus simple consiste à utiliser la fonction Helper global **redirect** :

```
Route::get('/dashboard', function () {  
    return redirect('home/dashboard');  
});
```

Redirect

Parfois, vous souhaitez peut-être rediriger l'utilisateur vers **son emplacement précédent**, par exemple lorsqu'un formulaire soumis n'est pas valide. Vous pouvez le faire en utilisant la fonction Helper globale **back**. Étant donné que cette fonctionnalité utilise la session, assurez-vous que la route appelant la fonction back utilise le groupe middleware **web** :

```
Route::post('/user/profile', function () {  
    // Validation de la demande...  
  
    return back();  
});
```

Back()

Lorsque vous appelez le Helper **redirect** sans paramètre, une instance de **Illuminate\Routing\Redirector** est renvoyée, vous permettant d'appeler n'importe quelle méthode sur l'instance **Redirector**.

Par exemple, pour générer un **RedirectResponse** vers une route nommée, vous pouvez utiliser la méthode **route** :

```
return redirect()->route('login');
```

Redirection vers des routes nommées

Si votre route a des paramètres, vous pouvez les passer comme deuxième argument à la méthode **route** :

```
// Pour une route avec l'URI suivant : /profile/{id}  
  
return redirect()->route('profile', ['id' => 1]);
```

Redirection vers des routes nommées

Vous pouvez également générer des redirections vers les actions du contrôleur.

Pour ce faire, passez le contrôleur et le nom de l'action à la méthode **action** :

```
use App\Http\Controllers\UserController;  
  
return redirect()->action([UserController::class, 'index']);
```

Redirection vers les actions du contrôleur

Parfois, vous devrez peut-être rediriger vers un domaine en dehors de votre application. Vous pouvez le faire en appelant la méthode **away**, qui crée un **RedirectResponse** sans aucun codage, validation ou vérification d'URL supplémentaire :

```
return redirect()->away('https://www.google.com');
```

Redirection vers des domaines externes

La redirection vers une nouvelle URL et le flashage des données vers la session sont généralement effectués en même temps. En règle générale, cela se fait après avoir effectué une action avec succès lorsque vous envoyez un message de réussite à la session. Pour plus de commodité, vous pouvez créer une instance **RedirectResponse** et envoyer des données **flash** à la session dans une seule chaîne de méthodes fluide :

```
Route::post('/user/profile', function () {  
    // ...  
  
    return redirect('dashboard')->with('status', 'Profile updated!');  
});
```

Redirection avec des données de session flashées

Une fois l'utilisateur redirigé, vous pouvez afficher le message flashé de la session .

Par exemple, en utilisant la syntaxe **Blade** :

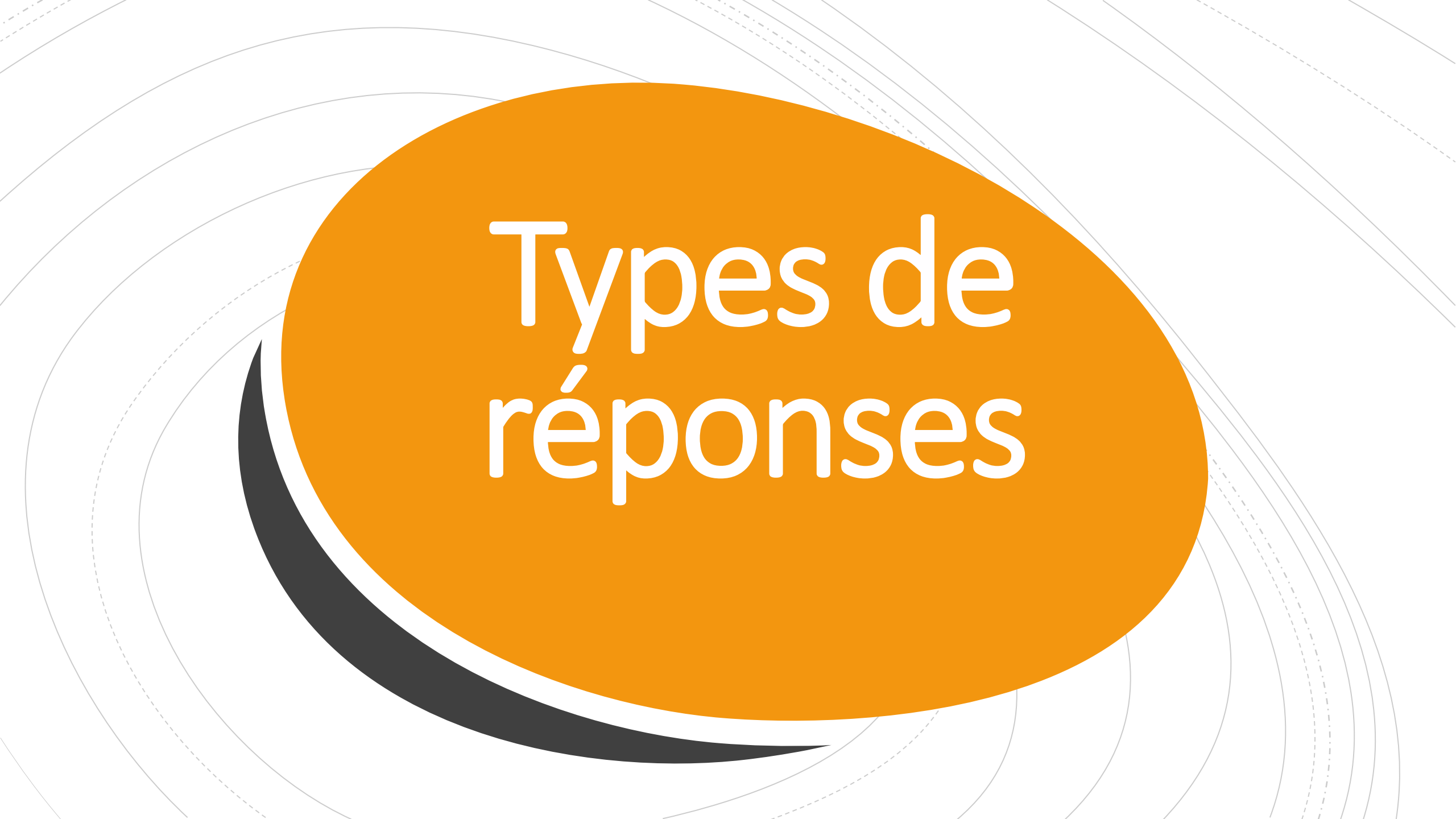
```
@if (session('status'))  
    <div class="alert alert-success">  
        {{ session('status') }}  
    </div>  
@endif
```

Redirection avec des données de session flashées

Vous pouvez utiliser la méthode **withInput** fournie par l'instance **RedirectResponse** pour flasher les données d'entrée de la demande actuelle dans la session avant de rediriger l'utilisateur vers un nouvel emplacement. Cela se fait généralement si l'utilisateur a rencontré une erreur de validation. Une fois l'entrée flashée à la session, vous pourrez facilement la récupérer lors de la prochaine requête pour repeupler le formulaire :

```
return back()->withInput();
```

Redirection avec entrée

The background features a series of concentric circles in light grey, some solid and some dashed, creating a ripple effect. A dark grey, curved swoosh is positioned on the left side, partially overlapping the orange oval.

Types de réponses

Le Helper **response** peut être utilisé pour générer d'autres types d'instances de réponse. Lorsque le Helper **response** est appelé sans arguments, une implémentation du contrat **Illuminate\Contracts\Routing\ResponseFactory** est renvoyée. Ce contrat fournit plusieurs méthodes utiles pour générer des réponses.

Si vous avez besoin de contrôler le statut et les en-têtes de la réponse, mais que vous devez également renvoyer une vue comme contenu de la réponse, vous devez utiliser la méthode **view** :

```
return response()  
    ->view('hello', $data, 200)  
    ->header('Content-Type', $type);
```

Bien entendu, si vous n'avez pas besoin de transmettre un code d'état HTTP personnalisé ou des en-têtes personnalisés, vous pouvez utiliser la fonction Helper globale **view**.

```
return view('hello');
```

Afficher les réponses

La méthode **json** définira automatiquement l'en-tête **Content-Type** sur **application/json**, ainsi que convertira le tableau donné en **JSON** à l'aide de la fonction **json_encode**:

```
return response()->json([  
    'name' => 'Abigail',  
    'state' => 'CA',  
]);
```

Réponses JSON

La méthode **download** peut être utilisée pour générer une réponse qui force le navigateur de l'utilisateur à télécharger le fichier au chemin donné. La méthode **download** accepte un nom de fichier comme deuxième argument de la méthode, qui déterminera le nom de fichier qui est vu par l'utilisateur téléchargeant le fichier. Enfin, vous pouvez passer un tableau d'en-têtes HTTP comme troisième argument à la méthode :

```
return response()->download($pathToFile);
```

```
return response()->download($pathToFile, $name, $headers);
```

Téléchargements de fichiers

La méthode **file** peut être utilisée pour afficher un fichier, tel qu'une image ou un PDF, directement dans le navigateur de l'utilisateur au lieu de lancer un téléchargement. Cette méthode accepte le chemin d'accès au fichier comme premier argument et un tableau d'en-têtes comme deuxième argument :

```
return response()->file($pathToFile);
```

```
return response()->file($pathToFile, $headers);
```

Fichier de réponses